Sumário

- Guia de execução, uso e testes · SGS Recife
  - Caminho rápido (8 comandos)
  - 1. Iniciar o projeto do zero
  - 2. Usar o sistema
  - 3. Testes manuais (roteiro validado)
  - 4. Testes automatizados
  - 5. Matriz de validação dos requisitos
  - 6. Problemas comuns
  - 7. Roteiro de demonstração (10 a 15 minutos)
  - 8. Comandos do dia a dia
  - 9. Como este guia foi validado (10/11/2026)

Guia de execução, uso e testes · SGS Recife

- **Versão de referência:** `main` em `4a07f71` (PR #4, 10/11/2026).
- **Para quem é:** quem sabe o básico de programação e nunca rodou o projeto.
- **Como foi validado:** cada comando e cada teste manual daqui foi executado num clone novo do repositório em 10/11/2026 (§ 9). O que não pôde ser verificado está marcado **[não verificado]**.
- **O que está implementado:** RELATORIO_IMPLEMENTACAO_SGS_RECIFE.md.

> **Ambiente de demonstração, não de produção.** As senhas do banco são de desenvolvimento, as chaves são geradas na sua máquina e o certificado HTTPS vem de uma CA local. Em produção (`APP_ENV=prod`) a API recusa segredos de exemplo, e o `seed demo` não roda.

Caminho rápido (8 comandos)

Se você já tem Git, Docker e uv instalados (§ 1.1):

```
git clone https://github.com/ViniciusVbm/sgs-api.git
cd sgs-api
uv sync --frozen
mkdir -p secrets
uv run python -m src.tools.jwt_keys    --output secrets/jwt-keys.json
uv run python -m src.tools.crypto_keys --output secrets/crypto-keys.json
docker compose -f docker/compose.yml up -d --build --wait
docker compose -f docker/compose.yml exec -e SGS_SEED_SENHA='escolha-uma-senha-de-dev' api sgs-admin
        seed demo
```

Depois:

1. Abra **https://localhost** e aceite o aviso de certificado.
2. Entre com o usuário **`demo.carla.dias`** e a senha que você escolheu.
3. Faça o cadastro do 2FA (§ 2.3).

As portas 80 ou 443 estão ocupadas, ou já existe outro `sgs-api` rodando? Veja o § 1.5 antes do `up`.

1. Iniciar o projeto do zero

1.1 Verificar as ferramentas

Rode no terminal; cada comando deve mostrar uma versão:

```
git --version              # qualquer versão recente
docker --version           # 26 ou mais nova
docker compose version     # v2 (o compose usa "subpath" de volume)
uv --version               # instala o Python 3.13 sozinho
```

Ferramenta	Obrigatória?	Para quê	Como instalar
Git	sim	baixar o código	https://git-scm.com/downloads

Ferramenta	Obrigatória?	Para quê	Como instalar
Docker com Compose v2	sim	rodar banco, API e proxy	https://docs.docker.com/get-docker/ (no Linux, o usuário precisa estar no grupo `docker` ou usar `sudo`)
uv	sim	gerar as chaves e rodar os testes	https://docs.astral.sh/uv/getting-started/installation/
Navegador (Chrome, Firefox, Edge)	sim	usar o sistema	—
Aplicativo autenticador (FreeOTP, Google ou Microsoft Authenticator)	não	códigos do 2FA no celular (há alternativa no terminal, § 2.3)	loja do celular
GNU Make	não	atalho `make check`	já vem no Linux e no macOS

A instalação das ferramentas em si **[não verificado]**: a validação partiu de uma máquina Linux que já as tinha.

1.2 Baixar o projeto

```
git clone https://github.com/ViniciusVbm/sgs-api.git
cd sgs-api
```

Todos os comandos seguintes são rodados dentro da pasta `sgs-api`.

1.3 Configuração: não é preciso criar `.env`

- **O ambiente Docker não usa o arquivo `.env`.** A configuração de desenvolvimento já está em `docker/compose.yml`.
- O `.env.example` só serve para quem quer rodar a API **fora** do Docker (instalacao.md).
- **O único preparo obrigatório são as duas chaves** da pasta `secrets/`, que fica fora do Git:

```
uv sync --frozen                                                    # instala as dependências
        (1ª vez demora)
mkdir -p secrets
uv run python -m src.tools.jwt_keys    --output secrets/jwt-keys.json      # chave que assina o login
uv run python -m src.tools.crypto_keys --output secrets/crypto-keys.json   # chave que cifra o 2FA no
       banco
```

Gere as chaves **uma vez só**. Se trocá-las depois, as sessões e os cadastros de 2FA existentes deixam de valer.

1.4 Iniciar o sistema

```
docker compose -f docker/compose.yml up -d --build --wait
```

- `-d` roda em segundo plano, `--build` monta a imagem da API e `--wait` só devolve o terminal quando tudo está pronto.
- **Tempo:** na primeira vez, alguns minutos, porque baixa as imagens e precisa de internet. Na validação, com as imagens já baixadas, levou **15 segundos**.

Para conferir se está pronto:

```
docker compose -f docker/compose.yml ps        # "healthy" em db e api; proxy "running"; db-certs
       "Exited (0)"
curl -k https://localhost/health/ready         # deve mostrar {"status":"ok"}
```

Serviço	O que é
`db-certs`	Gera os certificados TLS do banco e termina; é normal ficar `Exited (0)`
`db`	Banco PostgreSQL 17 (sem porta aberta para fora)
`api`	A API em Python; na subida, aplica as migrações do banco
`proxy`	Caddy: HTTPS, interface web e encaminhamento para a API

1.5 Portas ocupadas ou outro ambiente já rodando

O projeto usa as portas **80** e **443**. Se estiverem ocupadas, ou se já houver um ambiente `sgs-api` de **outra cópia** do repositório, use outras portas e outro nome de projeto (`-p`). Sem `-p`, o `up` **substitui** os contêineres `sgs-api` existentes.

```
export SGS_HTTP_PORT=8080 SGS_HTTPS_PORT=8443 SGS_CORS_ORIGINS=https://localhost:8443
docker compose -p sgs-local -f docker/compose.yml up -d --build --wait
```

- **Acesse:** `https://localhost:8443`.
- **`SGS_CORS_ORIGINS`** precisa ter **exatamente** esse endereço, senão a página volta ao login a cada recarga.
- **Repita o `-p sgs-local`** em todos os comandos `docker compose` (`ps`, `logs`, `exec`, `down`).
- **Não use `http://localhost:8080`:** o redirecionamento leva para a porta 443.

1.6 Endereços

O quê	Endereço (portas padrão)	Observação
Interface (front)	https://localhost/	
API	https://localhost/api/v1/...	sem login, responde 401
Saúde	https://localhost/health/ready	`{"status":"ok"}`
Contrato da API	`contracts/openapi.yaml` e `docs/api/endpoints.md`	55 operações
Swagger (`/docs`)	**não abre pelo endereço acima** (problema A1 do relatório); use o comando abaixo	
Banco	não exposto	`docker compose -f docker/compose.yml exec db psql -U sgs sgs`

Para abrir o Swagger (cria um atalho temporário só na sua máquina):

```
docker run --rm -d --name sgs-docs --network sgs-api_default -p 127.0.0.1:8001:8000 \
  alpine/socat tcp-listen:8000,fork,reuseaddr tcp-connect:api:8000
```

- **Abra:** http://127.0.0.1:8001/docs.
- **Para fechar:** `docker stop sgs-docs`.
- **Com `-p sgs-local`:** troque a rede para `sgs-local_default`.

1.7 Carregar os dados de demonstração

```
docker compose -f docker/compose.yml exec -e SGS_SEED_SENHA='escolha-uma-senha-de-dev' api sgs-admin
        seed demo
```

- **O que cria:** 37 unidades, 31 equipamentos, 35 serviços e **30 usuários** `demo.*`, todos fictícios.
- **Senha:** todos os usuários recebem a senha de `SGS_SEED_SENHA`.

- **Rodar de novo** não duplica nada **e não muda a senha** de quem já existe. Sem `SGS_SEED_SENHA`, o comando mostra uma "senha gerada" que só vale para usuários criados naquela execução (problema A2 do relatório).

1.8 Parar e voltar depois, sem perder dados

```
docker compose -f docker/compose.yml down                    # para; os dados ficam guardados
docker compose -f docker/compose.yml up -d --wait            # volta com os mesmos dados e
        usuários
```

Validado: depois de `down` e `up`, a mesma usuária entrou com a mesma chave 2FA e viu as mesmas 37 unidades.

> ⚠️ **`docker compose -f docker/compose.yml down -v` apaga o banco** (unidades, usuários, cadastros de 2FA) e os certificados. Use só para recomeçar do zero; depois, rode o seed de novo (§ 1.7).

2. Usar o sistema

2.1 Usuários de demonstração (um por papel)

Papel	Usuário	O que vê
Gestor de Dados (central)	`demo.carla.dias`	rede inteira; menu completo
Técnico Distrital (DS IV)	`demo.nicolas.brito`	lista abre no DS IV
Gestor de Unidade (CAPS II Espaço Azul)	`demo.sofia.barros`	começa na própria unidade
Coordenador Técnico (Saúde Mental)	`demo.gabriela.reis`	área técnica
Diretoria	`demo.yara.duarte`	Início, Unidades, Qualidade
Auditor	`demo.bento.sa`	Histórico (sem Início)
Administrador de TI	`demo.ana.lima`	papéis e usuários

A lista completa está em `seeds/demo/usuarios.json`. A senha é a do `SGS_SEED_SENHA`; **nunca use essas contas fora do ambiente local**.

2.2 Entrar

1. Abra https://localhost.
2. O navegador vai avisar que o certificado não é confiável: é o certificado local do Caddy. Clique em **Avançado** e em **Continuar**.
3. Preencha **Usuário** e **Senha** e clique em **Continuar**.

2.3 Primeiro acesso com 2FA (segundo fator)

No primeiro acesso de cada usuário, a tela **"Configure o segundo fator"** mostra:

- uma **chave** (texto);
- um **endereço** `otpauth://`.

Você tem duas opções:

- **Com celular:** no aplicativo autenticador, escolha "inserir chave manualmente", cole a chave e digite o código de 6 dígitos que aparecer.
- **Sem celular:** gere o código no terminal, dentro da pasta `sgs-api`:

```
uv run python -c "import pyotp; print(pyotp.TOTP('COLE_A_CHAVE_AQUI').now())"
```

Guarde a chave junto do nome do usuário. Ela aparece uma única vez e será usada em **todo** login seguinte, na tela "Confirme que é você". Cada código vale cerca de 30 segundos e não pode ser reaproveitado.

2.4 O que já funciona e o que não funciona

- **Funciona:**
  - login com 2FA, sessão (inclusive ao recarregar a página) e sair;
  - menu de acordo com o papel;
  - **Início** com os indicadores;
  - **Unidades**, com busca, filtros, ordenação e páginas.
- **Ainda não funciona** (aparece de propósito):
  - **"Tela em implantação":** ficha da unidade, cadastro e edição, serviços, qualidade, histórico, verificação da cadeia e tela de papéis. Serão as próximas entregas (T8 a T11).
  - **"em desenvolvimento":** solicitações e validações, notificações (o sino), importação, exportação e usuários. Ainda não existem na API.

Não teste esses itens como se fossem defeitos: é o comportamento esperado hoje.

3. Testes manuais (roteiro validado)

Use as portas padrão (ou as do § 1.5). São 22 passos, todos verificados em 10/11/2026 (§ 9):

- 17 repetidos por script no navegador;
- o C1 por comando;
- L6, U5, U6 e P4 pelos testes E2E, que exercitam a mesma tela.

3.1 Login, 2FA e sessão

Etapa	O que fazer	Resultado esperado
L1	Entrar com `demo.carla.dias` e uma senha **errada**	Mensagem "Usuário ou senha inválidos."; continua na tela de login
L2	Entrar com a senha certa (primeiro acesso)	Tela "Configure o segundo fator", com a chave e o endereço `otpauth://`, **sem** QR Code
L3	Digitar `000000`	Mensagem de erro no campo; continua na mesma tela
L4	Digitar o código certo (§ 2.3)	Abre o **Início**, com "Boa tarde, Carla." (ou bom dia/boa noite)
L5	Recarregar a página (F5)	Continua logada; não pede senha
S1	Clicar em **Sair** (ícone no canto superior direito) e depois recarregar	Volta ao login e continua no login
L6	Clicar em "Esqueci minha senha" na tela de login	Aviso para procurar o Administrador de TI

3.2 Início (painel)

Etapa	O que fazer	Resultado esperado
D1	Como Carla, olhar "Unidades cadastradas"	**37**, o mesmo total da API
D2	Clicar numa barra de "Unidades por distrito"	Abre a lista já filtrada por aquele distrito (endereço com `?distrito=`)

3.3 Lista de unidades

Etapa	O que fazer	Resultado esperado
U1	Menu **Unidades**	"1–20 de 37 unidades"; colunas com nome do tipo e do distrito
U2	Digitar `CAPS` em Buscar	Só unidades com "CAPS" no nome (6 no seed); o endereço passa a ter `?q=CAPS`
U3	Limpar a busca e clicar em **Próxima**	"21–37 de 37 unidades"
U4	Buscar `nao-existe`	"Nenhuma unidade encontrada." (estado vazio)
U5	Escolher um distrito e uma ordenação e recarregar a página	Os filtros continuam aplicados (ficam no endereço)
U6	Usar a busca do topo ("Buscar por nome ou CNES") a partir do Início	Abre a lista com a busca aplicada

3.4 Perfis e restrições de acesso

Etapa	O que fazer	Resultado esperado
P1	Entrar como `demo.yara.duarte` (Diretoria) e abrir `https://localhost/#/historico`	Tela "Sem acesso". A própria API também nega (403)
P2	Entrar como `demo.nicolas.brito` (Técnico Distrital) e abrir **Unidades**	Abre "Unidades do Distrito Sanitário IV" (4 unidades). Em Distrito, "Todos os distritos" mostra a rede inteira: é só um filtro de navegação
P3	Entrar como `demo.sofia.barros` (Gestora de Unidade)	Começa em "Minha unidade" (a ficha está "em implantação"); menu com 3 itens
P4	Comparar os menus dos 7 usuários do § 2.1	Cada papel vê só os itens das suas permissões (tabela no relatório, § 5)

3.5 Estados de erro, responsividade e segurança

Etapa	O que fazer	Resultado esperado
E1	Rodar `docker compose -f docker/compose.yml stop api` e abrir **Unidades** no navegador	Aviso de erro "Não foi possível concluir a operação." Depois rode `docker compose -f docker/compose.yml start api` e aguarde uns 10 s
R1	No navegador, abrir as ferramentas (F12), ativar o modo de dispositivo e escolher a largura **390 px**	A lista se ajusta, sem barra de rolagem horizontal
C1	`curl -skI https://localhost/ | grep -i content-security`	Mostra a política de segurança (CSP), sem `unsafe-eval`

O estado **"carregando"** aparece por frações de segundo e é difícil de ver à mão. Ele é provado pelo teste automatizado `tests/e2e_front/test_estados.py`.

4. Testes automatizados

O que são: programas que verificam o sistema sozinhos. Rode-os antes de enviar mudanças, ou para provar que algo funciona.

Conjunto	Comando	O que verifica	Precisa de	Tempo (validação)
Rápidos	`make check` (sem Make: veja abaixo)	estilo do código, tipos, arquitetura e ~980 testes de unidade, integração e regras do front	**Docker ligado** (cria um PostgreSQL temporário por teste)	~90 s

Conjunto	Comando	O que verifica	Precisa de	Tempo (validação)
Lentos, de infraestrutura	`uv run pytest -m slow --ignore=tests/e2e_front`	TLS do banco, entrega pelo Caddy, uso dos índices	Docker	~40 s
E2E do front	`uv run pytest tests/e2e_front -m slow`	o sistema inteiro no navegador: login, 2FA, perfis, Início, lista	Docker e Chrome (ou `uv run playwright install chromium`)	~2 min
Todos	`make test-all`	os três acima	Docker e navegador	**hoje termina com 1 erro conhecido** (problema A5 do relatório); prefira os comandos separados

- **Sem Make:** `make check` equivale a rodar em sequência `uv run ruff check .`, `uv run ruff format --check .`, `uv run mypy`, `uv run lint-imports` e `uv run pytest -m "not slow"`.
- **Os testes não usam o seu ambiente:** eles sobem os próprios contêineres, com nomes e portas próprias, e apagam tudo no fim.

Como ler o resultado:

- **Sucesso:** a última linha diz, por exemplo, `979 passed, 3 skipped` e não aparece `failed` nem `error`. Os 3 `skipped` são esperados e justificados.
- **Falha:** a última linha traz `failed` ou `error`. Logo acima, o resumo lista cada caso, por exemplo:

```
FAILED tests/e2e_front/test_lista.py::test_busca_e_busca_do_topo - AssertionError: ...
```

- **Para ver os detalhes de um teste:** rode só ele, com mais informação:

```
uv run pytest "tests/e2e_front/test_lista.py::test_busca_e_busca_do_topo" -m slow -vv
```

- **Para parar no primeiro erro:** acrescente `-x`. O `make test-all` também grava `reports/junit-test-all.xml`.

5. Matriz de validação dos requisitos

Requisitos da spec 012 (`specs/012-frontend/requirements.md`). Os testes manuais são os do § 3; os automatizados ficam em `tests/`.

5.1 Implementados e testáveis

Requisito	Funcionalidade	Como testar manualmente	Teste automatizado	Resultado esperado
R1.1	Front e API no mesmo endereço	abrir `/` e `/health/ready`	`tests/e2e/test_frontend_entrega.py`	página do sistema; `{"status":"ok"}`
R1.2	Visual do protótipo	comparar com o protótipo	`tests/e2e_front/test_aparencia.py`	mesmo visual
R1.3	Nada carregado de fora	F12 → Rede: todas as linhas de `localhost`	`test_aparencia.py`	nenhum domínio externo
R1.4	Celular (390 px)	R1	`test_aparencia.py`	sem rolagem horizontal
R1.6	Vazio e erro	U4, E1	`test_estados.py`, `test_lista.py`	mensagens do § 3
R2.1, R2.7	Login com "Usuário" e mensagens da API	L1, L4	`test_autenticacao.py`	"Usuário ou senha inválidos."

Requisito	Funcionalidade	Como testar manualmente	Teste automatizado	Resultado esperado
R2.2	2FA com chave em texto	L2	`test_autenticacao.py`	chave e `otpauth://`, sem QR Code
R2.3	Código do 2FA; token só em memória	L4; F12 → Aplicação → Armazenamento local só com `sgs-theme`	`test_autenticacao.py`	entra; nenhum token salvo
R2.4	Sessão ao recarregar	L5	`test_autenticacao.py`	continua logado
R2.6	Sair encerra a sessão	S1	`test_autenticacao.py` (4 casos, inclusive token expirado)	volta ao login, também depois de recarregar
R2.8	Esqueci a senha	L6	`test_autenticacao.py`	orientação para o Administrador de TI
R2.10	Sem contas ou senhas de demonstração na tela	olhar a tela de login	`tests/architecture/test_frontend_estatico.py`	nenhuma lista de contas
R3.1, R3.2	Menu por permissões (7 papéis)	P4	`test_perfis.py`	menus do relatório, § 5
R3.3, R3.4	"Sem acesso" e 403 da API	P1	`test_perfis.py`	"Sem acesso" e 403
R3.5	Lista do distrito e ficha própria	P2, P3	`test_perfis.py`, `test_lista.py`	DS IV; "Minha unidade"
R4.1	Início com indicadores da API	D1, D2	`test_inicio.py`	37; barra leva à lista
R5.1	Busca, filtros, ordenação, filtros no endereço	U2, U5, U6	`test_lista.py`	lista filtrada; o endereço guarda os filtros
R5.2	Paginação	U1, U3	`test_lista.py`	"21–37 de 37"
R10.1, R11.1, R14.1	Itens sem backend visíveis	abrir Validações, o sino, Importar	`test_perfis.py`	aviso "em desenvolvimento"
R15.1 a R15.4	Escape de HTML, CSP, armazenamento, sem `eval`	C1	`test_frontend_estatico.py`, `test_frontend_entrega.py`	CSP presente; testes verdes

5.2 Parcialmente implementados

Requisito	O que já existe	O que falta
R1.5 (acessibilidade)	avisos com `aria-live`, rótulos nos campos, `prefers-reduced-motion` do protótipo	o modal com foco preso ainda não é usado em nenhuma tela nem testado (T9, T13)
R1.6 (carregando)	existe e tem teste automatizado	difícil de ver manualmente
R2.5, R2.11	401 → renovar → repetir; login sem 2FA em dev	só testáveis de forma automatizada, com respostas simuladas no teste. O ambiente local exige 2FA
R13.1	menus e telas sem backend marcados	revisão de todas as 24 funcionalidades (T12)

5.3 Ainda não disponíveis (não testar)

- **Ficha da unidade, histórico e verificações:** R6.1 a R6.5, R9.1 (T8).
- **Edição e cadastro:** R7.1 a R7.6 (T9).
- **Serviços:** R8.1 (T10).

- **Qualidade, histórico geral e cadeia:** R12.1 a R12.4 (T11).
- **Dependem de backend ou de decisão:**
  - exportação: R5.4 e o R5.5 proposto;
  - filtros extras: R5.3;
  - solicitações: R10.2;
  - notificações: R11.2;
  - importação: R14.2;
  - redefinição de senha: R2.9;
  - indicadores de solicitações e regras: R4.2;
  - aparelhos: R9.2.

Requisitos do backend (specs 002, 003, 006, 008, 009, 011): estão implementados e cobertos pelos testes de integração do `make check`. A rastreabilidade completa está em REQUISITOS_EVIDENCIAS.md. Exemplos que dá para ver pela interface:

- o bloqueio por 15 minutos depois de 5 senhas erradas (`tests/integration/identity/test_login.py::test_cinco_falhas_bloqueiam_por_quinze_minutos`; **[não verificado] manualmente** nesta rodada, para não bloquear os usuários da demonstração);
- o limite de 10 logins por minuto por IP (testes em `tests/integration/identity/`).

6. Problemas comuns

Problema	Como diagnosticar	Solução mais simples
Docker não inicia ou "permission denied"	`docker ps`	inicie o Docker Desktop ou o serviço (`sudo systemctl start docker`); no Linux, adicione o usuário ao grupo `docker` e entre de novo **[não verificado]**
Porta ocupada (`port is already allocated`)	`ss -ltnp | grep -E ':(80|443) '`	§ 1.5
Outro `sgs-api` já rodando	`docker compose ls`	§ 1.5 (com `-p`), para não substituir o outro
A API não sobe: falta `secrets/...json`	`docker compose -f docker/compose.yml logs api`	gere as chaves (§ 1.3) e rode o `up` de novo
O banco não fica saudável	`docker compose -f docker/compose.yml logs db-certs db`	confira a internet (a 1ª subida baixa o `openssl`). ⚠️ Último recurso: `down -v` (apaga os dados)
Erro de migração	`docker compose -f docker/compose.yml exec api alembic current`	deve mostrar `0011_09 (head)`. Não rode o `alembic` fora do Docker
O front não abre	`curl -k https://localhost/health/ready` e `docker compose -f docker/compose.yml ps`	espere o `healthy`; rode os comandos dentro da pasta `sgs-api`
A API não responde	`docker compose -f docker/compose.yml logs --tail 50 api`	`docker compose -f docker/compose.yml restart api`
A página volta ao login a cada F5	`curl -sk -X POST https://localhost:8443/api/v1/auth/refresh -H 'X-Requested-With: sgs' -H 'Origin: https://localhost:8443' -o /dev/null -w '%{http_code}'` (use o seu endereço)	**403**: o endereço não está em `SGS_CORS_ORIGINS` (§ 1.5). **401** é o normal sem login
"Usuário ou senha inválidos."	`docker compose -f docker/compose.yml exec db psql -U sgs sgs -c "select count(*) from users where username like 'demo.%'"`	**0**: rode o seed (§ 1.7). **30**: a senha é a da **primeira** execução do seed

Problema	Como diagnosticar	Solução mais simples
"Muitas tentativas" ou usuário bloqueado	—	espere 1 minuto (limite) ou 15 minutos (5 senhas erradas)
Código 2FA recusado	confira o horário do computador e do celular	o código vale ~30 s e não pode ser repetido; espere o próximo
Perdi a chave 2FA	—	não há redefinição ainda. Use outro usuário `demo.*` ou, ⚠️ apagando tudo, `down -v` + `up` + seed
Não aparecem unidades	mesmo comando de `count`, trocando por `select count(*) from health_units`	0: rode o seed (§ 1.7)
Swagger abre a tela do sistema	`curl -sk https://localhost/openapi.json | head -c 40` (devolve HTML)	use o comando do § 1.6
Teste falhou	rode só o teste com `-vv` (§ 4)	leia a linha `E ...`; Docker ligado? Portas livres?
`make test-all` com 1 erro em `test_indices.py`	—	conhecido (A5); rode os conjuntos separados (§ 4)
Playwright sem navegador	`uv run pytest tests/e2e_front -m slow -x`	instale o Chrome ou rode `uv run playwright install chromium`

7. Roteiro de demonstração (10 a 15 minutos)

1. **Iniciar** (2 min): `docker compose -f docker/compose.yml up -d --wait` e `ps`. Mostre tudo saudável. Deixe o seed já rodado antes da apresentação.
2. **Acessar** (1 min): abra https://localhost e explique o certificado local.
3. **Login com 2FA** (2 min): faça L1 (senha errada) e L4 (código certo). Recarregue (L5) para mostrar que a sessão continua.
4. **Perfis** (3 min):
   - Carla (menu completo);
   - Nicolas (lista no DS IV);
   - Sofia (Minha unidade);
   - Yara em `#/historico` ("Sem acesso", e a API nega).
5. **Início** (2 min): explique os indicadores e clique numa barra de distrito.
6. **Unidades** (2 min): busca "CAPS", filtro, ordenação, página seguinte e busca sem resultado.
7. **Testes automatizados** (2 min): mostre o resultado do `make check` (979 aprovados). Ele leva ~90 s; rode antes ou ao vivo.
8. **O que está em desenvolvimento** (1 min): abra uma ficha ("Tela em implantação") e Validações ("em desenvolvimento"). Explique as próximas tarefas (T8 a T11) e o que depende do cliente.

Lista de verificação antes da apresentação:

- ☐ ambiente saudável (`ps`) e seed rodado com uma senha conhecida;
- ☐ as chaves 2FA de Carla, Nicolas, Sofia e Yara anotadas, ou os primeiros acessos já feitos;
- ☐ celular com o autenticador, ou o comando `pyotp` pronto;
- ☐ certificado aceito no navegador;
- ☐ saber distinguir o que funciona (§ 2.4) do que ainda não funciona.

8. Comandos do dia a dia

Para portas ou projeto alternativos, acrescente `-p <nome>` e as variáveis do § 1.5.

Ação	Comando	Apaga dados?
Subir	`docker compose -f docker/compose.yml up -d --build --wait`	não
Parar	`docker compose -f docker/compose.yml down`	não
Reiniciar a API	`docker compose -f docker/compose.yml restart api`	não
Reconstruir a API	`docker compose -f docker/compose.yml build api`	não
Ver logs	`docker compose -f docker/compose.yml logs -f api` (ou `db`, `proxy`)	não
Ver saúde	`curl -k https://localhost/health/ready`	não
Rodar as migrações	`docker compose -f docker/compose.yml exec api alembic upgrade head` (na subida em dev já rodam)	não
Carregar o seed	`docker compose -f docker/compose.yml exec -e SGS_SEED_SENHA='…' api sgs-admin seed demo`	não
Testes rápidos	`make check`	não
Testes completos	§ 4	não (usam ambientes próprios)
⚠️ Recomeçar do zero	`docker compose -f docker/compose.yml down -v`	**sim**: banco, usuários, 2FA e certificados

9. Como este guia foi validado (10/11/2026)

Ambiente:

- máquina Linux com Docker 29.9, Compose v5.6, uv 0.13 e Git 2.56;
- clone novo de `https://github.com/ViniciusVbm/sgs-api.git` (`4a07f71`);
- projeto `-p sgs-guia` e portas 8080 e 8443, porque 80 e 443 estavam ocupadas por outro ambiente, que foi preservado.

O que foi executado:

- preparo (§ 1.2 e § 1.3) e subida em 15 s, com tudo saudável;
- seed, inclusive a segunda execução (A2);
- endereços (§ 1.6) e o atalho do Swagger;
- um roteiro automatizado no navegador que repete o § 3 (L1 a L5, S1, D1, D2, U1 a U4, R1, E1, P1 a P3): **17 de 17 passos aprovados**;
- `down` e `up` sem `-v`, com os dados preservados;
- `make check` com 979 aprovados;
- testes lentos com 56 aprovados e 1 erro conhecido (A5), e separados com 12 + 45 aprovados;
- C1 (cabeçalhos de segurança).

Verificados pelos E2E, e não pelo script do roteiro: L6, U5, U6 e P4 (`test_autenticacao.py`, `test_lista.py`, `test_perfis.py`, aprovados na mesma rodada).

[Não verificado]:

- as portas padrão 80 e 443 nesta máquina (cobertas pelo CI `compose-smoke`);
- Windows e macOS;
- a instalação das ferramentas;
- o bloqueio por 5 senhas erradas, feito manualmente.